

Technical Debt Areas

1. `server.js` as a platform monolith

The main server file owns:

- middleware setup
- Socket.IO
- route mounting
- static web and store hosting
- health checks and policy pages
- Mongo bootstrap
- scheduler startup

This centralization makes changes easy in the short term but increases coupling and review risk.

2. Role and permission duplication

Permission rules appear in:

- `middleware/permissions.js`
- `models/permissions.model.js`
- `config/livestreamPermissions.js`
- user role fields themselves

This increases drift risk.

3. Read paths with side effects

Examples:

- post approval pending reads can backfill rows and notify approvers
- some counts are repaired during normal reads

This makes behavior harder to reason about and cache.

4. Mixed legacy and current data shapes

User roles, notification target handling, and some community fields show signs of incremental evolution rather than a single stable model.

5. Global process coupling

`global.io` and global helper hooks are pragmatic, but they hide dependencies and complicate testing and modularization.

Recommended cleanup order

1. unify RBAC and role resolution
2. isolate realtime and job concerns from `server.js`
3. remove side effects from read endpoints
4. normalize legacy fields and document migration paths